

Cairo University
Faculty of Engineering



Von Neumann RISC Processor Design

Team 9

CMPS301

To: Eng. Sherif

By:

Ahmed Salah Salem	1230008
Ahmed Mohammed Ahmed AbdelJaleel	1230012
Abdelrahman Essam Mohammed Ismail	1230057
Omar Ahmed Raafat	1230072

Table of Contents

Table of Contents.....	2
Instruction Format.....	3
Overview.....	3
Instruction Formats.....	3
F0 – No Operand.....	3
F1 – One Operand.....	3
F2 – Register Operations.....	3
F3 – Immediate / Branch.....	3
General Rules.....	3
Control Unit Design.....	4
Control Word.....	4
Multicycle Sequencing.....	4
Priority Rules.....	4
Pipeline Design & Registers.....	5
Stage Responsibilities.....	5
Fetch.....	5
Decode.....	5
Execute-1.....	5
Execute-2.....	5
Memory.....	5
Write-Back.....	5
Pipeline Registers.....	5
Pipeline Hazards.....	6
Data Hazards.....	6
Special Cases.....	6
Load-Use Hazard.....	6
Structural Hazards.....	6
Control Hazards.....	6
Misprediction Handling.....	6
Special Control Cases.....	6
5. Dynamic Branch Prediction.....	7
Predictor Type.....	7
Operation.....	7
In Fetch Stage.....	7
In Execute-2 Stage.....	7
Update Mechanism.....	7
Benefits.....	7
Misprediction Cost.....	7

Instruction Format

Overview

- Fixed 32-bit instruction word
- Memory: 4*4KB (12-bit address space)
- Core fields:
 - opcode (5 bits)
 - rdst, rsrc1, rsrc2 (3 bits each)
 - imm/offset (up to 16 bits)
 - Jmpflags(2 bits)

Instruction Formats

F0 – No Operand

- Structure: opcode and all zeros
- Used by: NOP, HLT, SETC, RET, RTI

F1 – One Operand

- Register mirrored in rdst and rsrc1
- Used by: NOT, INC, IN, OUT, PUSH, POP

F2 – Register Operations

- 2-register (F2-2R): MOV, SWAP
- 3-register (F2-3R): ADD, SUB, AND

F3 – Immediate / Branch

- Branch (Addr12):
 - 12-bit target address
 - Early decode bits:
 - instr[16]: unconditional branch
 - instr[17]: conditional branch
- Immediate formats:
 - LDM, IADD, LDD, STD
- Interrupt format:
 - INT uses [1:0] index encoding

General Rules

- Immediate values stored in [15:0]
- Branch addresses stored in [11:0]
- Internal micro-ops (SWAP2, INT2, INT3) are not useable in assembler

Control Unit Design

Control Word

- **Execution & ALU**
 - ALU_OP
 - UPDATE_FLAGS
 - LOAD_FLAGS
- **Memory Control**
 - MEMR, MEMW
 - MEM_ADDRESS_SEL
 - MEM_WRITE_SEL
- **Register & I/O**
 - REG_WB_EN
 - OUTPUT_PORT_EN
- **Control Flow**
 - PC_WRITE_EN
 - COND_BRANCH
 - JMP_FLAG_SEL
- **Pipeline Control**
 - MULTICYCLE_STALL
 - MULTICYCLE_SEL
 - SWAP_2ND_CYCLE

Multicycle Sequencing

- Complex instructions broken into internal steps:
 - SWAP -> SWAP2
 - INT -> INT2 -> INT3
 - RTI -> RET
- Decode stage injects internal operations while stalling fetch

Priority Rules

- **PC-writing instructions have highest priority**
 - Stall other instructions
 - Flush pipeline on commit

Pipeline Design & Registers

Stage Responsibilities

Fetch

- Instruction fetch + next PC
- Early branch decoding and detection
- Branch prediction

Decode

- Instruction decode + register read
- Control signal generation
- Multicycle handling

Execute-1

- ALU operations
- Data forwarding applied

Execute-2

- Branch resolution, memory address calculation and branch misprediction detection

Memory

- Memory access
- Stack operations

Write-Back

- Register and flag updates

Pipeline Registers

- **IF_ID**
 - instr (32)
 - next_pc (32)
 - branch_prediction
- **ID_EX1**
 - Control word
 - Data:
 - register values (32-bit)
 - imm_offset (16)
 - register indices (3-bit)
 - next_pc (32)
- **EX1_EX2**
 - ALU outputs:
 - alu_result (32)
 - alu_flags (3)
 - Control signals
- **EX2_MEM**
 - Memory address/data (32-bit)
 - Flags and control signals
- **MEM_WB**
 - Write-back data (32-bit)
 - Flags (3-bit)
 - Register index

Pipeline Hazards

Data Hazards

- Forwarding paths:
 - EX1/EX2 → EX1
 - EX2/MEM → EX1
 - MEM/WB → EX1
- Forwarding to:
 - ALU inputs
 - Branch flags

Special Cases

- SWAP2 disables certain forwarding

Load-Use Hazard

- 1-cycle stall
- Up to 2 cycles if dependency persists

Structural Hazards

- Memory arbitration:
 - Memory stage gets priority
 - Fetch is stalled (FETCH_MEMORY_HAZARD)

Control Hazards

Misprediction Handling

- Flush:
 - IF/ID
 - ID/EX1
 - EX1/EX2
- Correct PC:
 - Target (if wrong-not-taken)
 - Next PC (if wrong-taken)

Special Control Cases

- PC-write instructions:
 - Stall pipeline
 - Flush on completion

5. Dynamic Branch Prediction

Predictor Type

- 2-bit counter

Operation

In Fetch Stage

- Early detection using:
 - instr[16] -> unconditional branch
 - instr[17] -> conditional branch
- Predictor decides:
 - Taken / Not taken

In Execute-2 Stage

- Actual branch result computed
- Compared with prediction

Update Mechanism

- Counter updated based on correctness:
 - Strong/Weak Taken
 - Strong/Weak Not Taken

Benefits

- Reduces pipeline stalls
- Improves instruction throughput

Misprediction Cost

- Flush 3 pipeline stages
- Correct PC injected